Multi-threaded Web Server Technical Documentation

Made by: Bernardo Reis 126004 | Fernando Santos 124808

In this document, we will explain every software decision in detail as well as display performance analysis.

Introduction

First, let's start with a little contextualization. Our task was to implement a web server, purely using C, with the knowledge acquired (and some not) from our classes. This implementation uses multi-process architecture in a thread-safe environment, customizable settings, keep-alive, log rotation with compression, LRU cache with DLL, HTTPS, and some other acquired information.

Server starting in main.c

When the user starts the server using ./server, the script main.c is the first who gets called, calling the function load_config from the config.c. The structure of the configuration can be seen below:

```c
typedef struct {
    int port;
    char document_root[256];
    int num_workers;
    int threads_per_worker;
    int min_threads;
    int max_threads;
    int max_queue_size;
    char log_file[256];
    int cache_size_mb;
    int timeout_seconds;
    int max_log_size;
} server_config_t;
```

The port is the server port, the document_root is the relative path to the server files folder (index.html, etc...), the num_work is the number of workers, which will be explained later as well as threads_per_worker, min_threads, max_threads and max_queue_size. The log_file variable contains the relative location to the log file from the project folder. The logging procedure will also be explained in a dedicated chapter. The cache_size_mb is the max cache size per worker, the timeout_seconds is used as the keep-alive timeout, and the max_log_size is the maximum size of the log.

All these variables are obtained from the file server.conf, which can, by default, be found in the project folder. After that, we create the shared memory, using the function made in shared_mem.c. The shared memory structure can be seen below:

```c
typedef struct {
    server_stats_t stats;
    FILE* log_file;
    cache_t* cache;

} shared_data_t;
```

This 3 structures will be explained in the next chapters, but before that, we will do a brief introduction to them: server_stats_t is the structure to the server statistics (status codes, bytes sent, peak of active connections), log_file is the pointer to the file descriptor of the log file and cache is the variable with the cache_t structure, that uses a DLL (Doubly Linked List) with hashing. This structure can already be used as a silhouette of how our multi-process architecture will be applied, since this structure is the core to parallel processing using the same variables. With this structure initialized, the program initializes POSIX semaphores that have this structure:

```c
typedef struct {
    sem_t* empty_slots;
    sem_t* filled_slots;
    sem_t* queue_mutex;
    sem_t* stats_mutex;
    sem_t* log_mutex;
    sem_t* cache_mutex;
} semaphores_t;
```

It was also possible to add each semaphore to its designated structure, but the approach used provides a cleaner code. The implementation of semaphores solves the race condition problems by ensuring mutual exclusion when accessing shared resources. This avoids data corruption and ensures consistency.

After that, the server opens the log file in append mode, to add new log information. This log file stays open until the closure of the server.

```c
shared_mem->log_file = fopen(conf->log_file, "a");
if (shared_mem->log_file == NULL) {
    fprintf(stderr, "Error opening log file!\n");
    exit(1);
}
```

Since this file could potentially be accessed by multiple threads at the same time and lead to a race condition, we had to define it inside the shared memory. The same happens to the cache, which is also initialized.

After all initializations, we display a success message, telling the user the server structure had successfully started. Succeeding this print, the server runs the function run_master, which starts the master process.

Master and Workers Introduction (master.c)

This relationship pretends to assimilate the producer-consumer solution, but since the master cannot directly access the worker's memory space, it cannot enqueue connections itself. Instead, the master acts as a producer by sending accepted client file descriptors to the worker's process via Unix Domain Sockets (UDS). The dispatcher thread in the worker process then receives these file descriptors and enqueues them into the local queue, which the worker threads (consumers) can process. This design preserves the producer-consumer pattern across process boundaries using inter-process communication.

In the beginning of the master file, after the includes, there are two important features: signal handling and a thread structure definition. The signal handling contains an atomic variable: keep_running. This variable is set to 0 when the user pretends to close the server, displaying a message informing the user the server is trying to do it, to achieve a graceful shutdown. This signal handling mechanism works thanks to the <signal.h> library. Going back to the other feature, the thread structure can be seen in the image below:

```
typedef struct
{
    shared_data_t *data;
    semaphores_t *semaphores;
    int delay;
} stats_thread_args;
```

This structure needs access to shared data and semaphores, since accessing the statistics (which is an attribute of the shared memory) can lead to race conditions. The delay is the number of seconds without the statistics terminal display.

Going directly to the function run_master, firstly we configure and clear the signals the user can use. After that we call a function called create_server_socket:

```c
int create_server_socket(int port)
{
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0)
        return -1;
    int opt = 1;
    setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    struct sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(port);
    if (bind(sockfd, (struct sockaddr *)&addr, sizeof(addr)) < 0)
    {
        close(sockfd);
        return -1;
    }
    if (listen(sockfd, 128) < 0)
    {
        close(sockfd);
        return -1;
    }
    return sockfd;
}
```

As explained in the beginning of this chapter, the implementation of Unix Domain Sockets was essential to our results. The function defined in the image above creates the server socket, binding this process to all available network interfaces.

Following the creation of the server socket, we define an array of *pids* (process ids) which is after used to store the ids of the child processes generated by master (the parent in this case) and log into the log file that the server started. We initialize a stats_thread_args structure and then create a thread to run in parallel displaying the stats in the terminal while the server runs. The delay in this case is 30 seconds. The socketpair procedure (sv) is necessary since we are using UDS.

With that code section explained, we enter the core of the project, the worker processes creation and connection handling.

For the worker processes creation, we initialize a loop of n_workers iterations and then use fork() to generate processes while knowing its *pids*. Then we enter a classic section of fork usage: the distinction between the parent process and the child. The child process enters its section (*pid* == 0), closing all the connection listeners, since the child processes don't accept connections nor distribute work. Then we initiliaze the SSL library, since we are using HTTPS, so the SSL protocol is a need. With SSL properly initialized, we finally initialize a worker_args structure, which has this format:

```
typedef struct
{
    shared_data_t *data;
    semaphores_t *semaphores;
    int worker_id;
    int channel_fd;
    server_config_t configs;
    SSL_CTX *ssl_ctx;
} worker_args;
```

All these variables will be sent to the worker as an argument. After that, here is an important function called: start_worker, that contains the worker_args argument and makes the child process a worker process. To maintain a structured documentation, we will already explain what happens to the child when the worker function ends: the program tries to close all variables that have allocated memory to avoid memory leaks. With all allocated memory freed, the child process exits successfully.

With the parent, we store the child *pid* in the array named worker_pids previously introduced as the "array of *pids*" and configure two other relevant connection handling configurations: we close sv[1], since it's the worker's end,

to avoid resource leaks. We use sv[0] since this is where the master procedure can send the file descriptors to the workers via UDS.

Then, the master executes a round robin algorithm while the atomic variable keep_running is set to 1, sending client connections to the workers as accepted file descriptors. The rest of this round robin is signal handling, to avoid an abrupt shutdown and ending with the function send_fd. This function is defined in uds.c and basically allows, with a set of specific configurations, to send the accepted file descriptor to the workers, something a solution without UDS couldn't easily achieve.

When the server is starting the shutdown procedure the master process exits its loop and, to avoid making some child processes orphan or even zombies, waits for all of them to exit and only after, prints to the user and in the log file that the server successfully stopped and ends its procedure in the master process, going back to the caller being terminated there.

Workers (worker.c)

This function also starts with a signal handler to send to, the further explained, thread pool the shutdown signal and avoid deadlocks. It's also declared a global thread pool, since the signal handling procedure is not local and cannot have our own arguments. In the start_worker function, after some signal handling configurations, we find this extensive line:

```
thread_pool_t *thread_pool = create_thread_pool(worker_arg.data, worker_arg.semaphores, worker_arg.worker_id, worker_arg.channel_fd, worker_arg.configs, worker_arg.ssl_ctx);
```

Which has all relevant variables to correctly initialize a structure to use a thread pool for each worker.

```c
typedef struct thread_pool
{
    pthread_t *threads;
    int num_threads;
    int min_threads;
    int max_threads;
    volatile _Atomic int shutdown;
    queue_t queue;
    shared_data_t *data;
    semaphores_t *semaphore;
    int worker_id;
    int channel_fd;
    server_config_t configs;
    SSL_CTX *ssl_ctx;
} thread_pool_t;
```

Almost all variables here are already explained, except the first 4 and the sixth. We need a threads pointer, since we dynamically create a thread array. To grant some safety boundaries, we also created minimum threads and maximum threads numbers. The queue variable contains the connection queue, and as this structure shows, the connection queue is local, and its structure can be seen in the underneath image:

```c
typedef struct
{
    int *fd_s;
    int max_size;
    int front;
    int rear;
    int count;
    pthread_mutex_t mutex;
    pthread_cond_t cond;
} queue_t;
```

This queue, since it can suffer changes in its properties (size, file descriptor arrays...) needs a mutex and a cond. With the thread pool executing, the worker waits for all the threads to be terminated and after that destroy the thread pool and return to the caller.

Thread Pool (thread_pool.c)

The function create_thread_pool starts the thread pool after successfully creating num_threads threads and a specific thread that works as a dispatcher. First, let's explain how the dispatcher works and why we need it.

The dispatcher thread is responsible for receiving client connections from the master process, enqueueing them into the worker process' local queue. This was supposed to be the master's job, but since we have different processes, the master cannot directly access or modify the worker's local queue due to process isolation. So, after receiving the client file descriptor with the function recv_fd, a function made in uds.c, the file descriptor gets set up by the protocol SSL, and if the SSL couldn't accept, then it happened an internal server error, which is returned to the client. Also, if the queue was full, the server also returns internal server error. Otherwise, the queue still has empty slots. After enqueueing the connection, the server adjusts the number of threads (with a max of 100 scalings) to try to make the dequeuing procedure faster. If the queue load is higher than 70%, this process creates a new thread to the thread pool. If the queue load is lower than 20%, this process removes one thread if any of them is not working.

While this enqueueing process is happening, the thread pool is already running in background, waiting for a client file descriptor to be enqueued. When it finally passes through the pthread_cond_wait, firstly it checks if the server has already ended or not to grant a graceful shutdown. If the shutdown was not triggered, the thread dequeues the first element of the queue. After that, we implement a timeout mechanism, waking the thread frequently to check if the server is shutting down. This ensures constant verification to keep-alive connections. After that, the worker initializes SSL, to read the encrypted data, and we use the method getpeername to get the client's IP address and print it on the log. After that, while the connection has keep-alive (every client connection starts with a keep-alive), and the client didn't end the connection, the worker thread enters a loop where it waits for a maximum of 5 seconds

for the client to send a new request (Idle Timeout). If data arrives within that window, the worker then switches to a TIMEOUT_SECONDS seconds timeout (Request Timeout) to allow the full request body to be received, processed, and responded before returning to the idle state. If the request has HTTP version 1.0, the connection is closed immediately after the response unless the client explicitly requested “Connection: keep-alive”, whereas HTTP 1.1 requests remain open by default until the client sends 'Connection: close' or the idle timeout expires.

```c
if (bytes_read <= 0)
{
    int err = SSL_get_error(ssl, bytes_read);

    if (err == SSL_ERROR_WANT_READ || err == SSL_ERROR_WANT_WRITE)
    {
        pthread_mutex_lock(&pool->queue.mutex);
        should_shutdown = pool->shutdown;
        pthread_mutex_unlock(&pool->queue.mutex);

        if (should_shutdown)
            break;

        continue;
    }
    break;
```

The image above shows another important edge case handling. It’s important to make sure that the connections were successfully read, otherwise the next procedures could cause a major memory leak. With a valid request, we finally can start parsing the http/https requests. Firstly, we create this structure:

```c
typedef struct
{
    char method[16];
    char path[512];
    char version[16];
    int connection_close;
} http_request_t;
```

This structure contains all necessary information we need extracted from the request, which is what the function parse_http_request does. Then, the program directly checks if the request path is "/api/stats", since we implemented a small API to our statistics dashboard website. This early verification saves time for the API request, since time is key. With that specific example, we will explain how our server handles the http requests and how it returns a response to the client. First, since we are accessing shared memory variables, we have to place semaphores in order to avoid race conditions. Since this is an API request, returning a JSON seems to be the most solid option. After building the response, with the help of the method snprintf, the server sends the response back to the caller (in this case the caller is the JavaScript of the statistics dashboard .html file) with the function send_http_response. This response is sent via SSL to keep the connection safe and encrypted. In this case, since it's an API Request without the need of reading an HTTP, we didn't use another function from the http.c file: read_http. After the response is sucessfully sent, the server locks the stats_mutex to add the outcome to the server statistics and unlocks after.

However, the example above is the simplest HTTP request the server handles, since the Content-Type is already known from the server, and there is no need to read files. If the request is not directed to the API, it is requesting a file, so we added a variable named pathBuffer, a String, that will contain the path to the requested file. First, the server should build the path String. If the requested path is the root directory (/) or any other valid directory (this is requested in the Announcement of Project Proposal file), serve the index.html

file. Otherwise, the server checks if the file path is valid. If the file is invalid, the server tries to return a 404-status code and return the 404-html file. If that file is also invalid, the server returns “Internal Server Error” (status 500). Inside each condition, we define the status, the status message, and the Content-Type. For valid files, the script first must check the content type, because there are multiple types of files that the server supports and contains. After that, the server checks if the request method was HEAD. If it was, there is no need to read the requested file; otherwise, the server must read the file content. As already introduced earlier, the server has a cache, that can store CACHE_SIZE_MB *NUM_WORKERS megabytes. Once this limit is reached, and the cache must store a new resource that’s still not there, it automatically evicts the least recently used (LRU) resource(s), until it has enough space to store. The cache.c file contains the relevant methods for our server cache, with some abstraction, but to see the concrete details of the cache (with a low-level abstraction) check dll.c and hash.c. The file cache operation is:

1. If the cache was initialized correctly, try to get the file searching for pathBuffer in the cache:
    a. If the file was found:
        i. Copy the file content in the cache to a local variable (body) and calculate the length of the content (bodylength).
        ii. Set cache_hit to 1, to avoid unnecessary operations.
    b. If the file was not found in the cache (cache_hit == 0), and the file is not null (because some files can be empty):
        i. Read the http with the function read_http from http.c to a local variable (body) and calculate the length of the content (bodylength).
        ii. Add the file content to the cache with the method cache_put
    c. The file is empty:
        i. The length of the content is 0. (bodylength = 0)
2. If the cache was not initialized:
    a. Read the http with the function read_http from http.c to a local variable (body) and calculate the length of the content (bodylength).

This reasoning allows us to have a faster server response and to obtain the file content. With all relevant information gathered, the server sends back to the client the http response, and after logs that information to the log file.

Then, the program updates all the relevant statistics. If the connection is not keep-alive or the connection timed out (either in the user idle wait phase or the active request transmission), the worker thread breaks the loop, performs a graceful SSL shutdown to secure the channel, frees the SSL resources, and finally closes the client socket before returning to the thread pool to wait for the next assignment.

The thread_pool.c file also contains two other functions: create_thread_pool, which was already explained in the worker.c, and the destroy_thread_pool, that stops all the active connections, clears the queue, and demands to all threads to exit (dispatcher included).

Log file and its rotation (logger.c)

Log rotation is a fundamental step to having a concise and organized log file. So, with the usage of the function "system" and the Unix command "logrotate", it was possible to make this process automatic. Every time the server starts or once in a hundred executions of the other logging function, the log file gets checked to analyze if the maximum log file was exceeded. If it exceeds 10MB, the log file gets rotated to the old_logs directory and compressed. However, the log file location may vary, thus, to help this implementation, when the log file is about to be rotated, the generate_logrotate_conf function gets called, to generate a valid configuration to be used by the logrotate command.

Server Cache (cache.c, dll.c and hash.c)

To optimize performance and minimize disk latency, the server employs an LRU (Least Recently Used) caching system. This mechanism relies on a hybrid data structure: a Hash Table is used for instant O(1) file retrieval, while a Doubly Linked List maintains the chronological order of file access. Every time a resource is accessed via cache_get, it is promoted to the head of the list, marking it as "recently used". When the server needs to cache a new file but lacks sufficient memory, the cache_put function triggers an eviction process. It identifies and removes items from the tail of the list—which represents the data accessed furthest in the past—until enough space is cleared. This strategy ensures that the most popular content remains in memory ("hot" data) while efficiently discarding data that is no longer needed.